

Knapsack problem

The **knapsack problem** is the following problem in combinatorial optimization:

Given a set of items, each with a weight and a value, determine which items to include in the collection so that the total weight is less than or equal to a given limit and the total value is as large as possible.

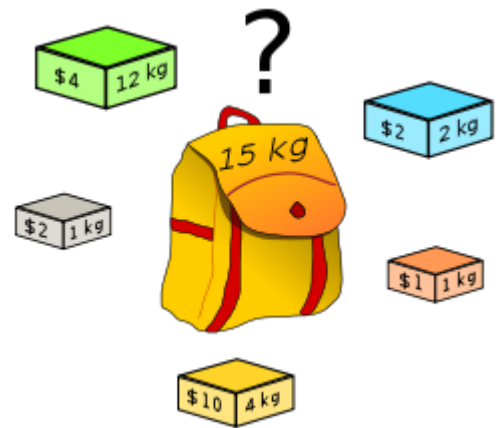
It derives its name from the problem faced by someone who is constrained by a fixed-size knapsack and must fill it with the most valuable items. The problem often arises in resource allocation where the decision-makers have to choose from a set of non-divisible projects or tasks under a fixed budget or time constraint, respectively.

The knapsack problem has been studied for more than a century, with early works dating as far back as 1897.^[1] The name "knapsack problem" dates back to the early works of the mathematician Tobias Dantzig (1884–1956),^[2] and refers to the commonplace problem of packing the most valuable or useful items without overloading the luggage.

Applications

Knapsack problems appear in real-world decision-making processes in a wide variety of fields, such as finding the least wasteful way to cut raw materials,^[3] selection of investments and portfolios,^[4] selection of assets for asset-backed securitization,^[5] and generating keys for the Merkle–Hellman^[6] and other knapsack cryptosystems.

One early application of knapsack algorithms was in the construction and scoring of tests in which the test-takers have a choice as to which questions they answer. For small examples, it is a fairly simple process to provide the test-takers with such a choice. For example, if an exam contains 12 questions each worth 10 points, the test-taker need only answer 10 questions to achieve a maximum possible score of 100 points. However, on tests with a heterogeneous distribution of point values, it is more difficult to provide choices. Feuerman and Weiss proposed a system in which students are given a heterogeneous test with a total of 125 possible points. The students are asked to answer all of the questions to the best of their abilities. Of the possible subsets of problems whose total point values add up to 100, a knapsack algorithm would determine which subset gives each student the highest possible score.^[7]



Example of a one-dimensional (constraint) knapsack problem: which boxes should be chosen to maximize the amount of money while still keeping the overall weight under or equal to 15 kg? A multiple constrained problem could consider both the weight and volume of the boxes.

(Solution: if any number of each box is available, then three yellow boxes and three grey boxes; if only the shown boxes are available, then all except for the green box.)

A 1999 study of the Stony Brook University Algorithm Repository showed that, out of 75 algorithmic problems related to the field of combinatorial algorithms and algorithm engineering, the knapsack problem was the 19th most popular and the third most needed after suffix trees and the bin packing problem.^[8]

Definition

The most common problem being solved is the **0-1 knapsack problem**, which restricts the number x_i of copies of each kind of item to zero or one. Given a set of n items numbered from 1 up to n , each with a weight w_i and a value v_i , along with a maximum weight capacity W ,

$$\begin{aligned} &\text{maximize } \sum_{i=1}^n v_i x_i \\ &\text{subject to } \sum_{i=1}^n w_i x_i \leq W \text{ and } x_i \in \{0, 1\}. \end{aligned}$$

Here x_i represents the number of instances of item i to include in the knapsack. Informally, the problem is to maximize the sum of the values of the items in the knapsack so that the sum of the weights is less than or equal to the knapsack's capacity.

The **bounded knapsack problem (BKP)** removes the restriction that there is only one of each item, but restricts the number x_i of copies of each kind of item to a maximum non-negative integer value c :

$$\begin{aligned} &\text{maximize } \sum_{i=1}^n v_i x_i \\ &\text{subject to } \sum_{i=1}^n w_i x_i \leq W \text{ and } x_i \in \{0, 1, 2, \dots, c\}. \end{aligned}$$

The **unbounded knapsack problem (UKP)** places no upper bound on the number of copies of each kind of item and can be formulated as above except that the only restriction on x_i is that it is a non-negative integer.

$$\begin{aligned} &\text{maximize } \sum_{i=1}^n v_i x_i \\ &\text{subject to } \sum_{i=1}^n w_i x_i \leq W \text{ and } x_i \in \mathbb{Z}, x_i \geq 0. \end{aligned}$$

One example of the unbounded knapsack problem is given using the figure shown at the beginning of this article and the text "if any number of each box is available" in the caption of that figure.

Computational complexity

The knapsack problem is interesting from the perspective of computer science for many reasons:

- The decision problem form of the knapsack problem (*Can a value of at least V be achieved without exceeding the weight W ?*) is NP-complete, thus there is no known algorithm that is both correct and fast (polynomial-time) in all cases.
- While the decision problem is NP-complete, the optimization problem is not, its resolution is at least as difficult as the decision problem, and there is no known polynomial algorithm which can

tell, given a solution, whether it is optimal (which would mean that there is no solution with a larger V , thus solving the NP-complete decision problem).

- There is a pseudo-polynomial time algorithm using dynamic programming.
- There is a fully polynomial-time approximation scheme, which uses the pseudo-polynomial time algorithm as a subroutine, described below.
- Many cases that arise in practice, and "random instances" from some distributions, can nonetheless be solved exactly.

There is a link between the "decision" and "optimization" problems in that if there exists a polynomial algorithm that solves the "decision" problem, then one can find the maximum value for the optimization problem in polynomial time by applying this algorithm iteratively while increasing the value of k . On the other hand, if an algorithm finds the optimal value of the optimization problem in polynomial time, then the decision problem can be solved in polynomial time by comparing the value of the solution output by this algorithm with the value of k . Thus, both versions of the problem are of similar difficulty.

One theme in research literature is to identify what the "hard" instances of the knapsack problem look like,^{[9][10]} or viewed another way, to identify what properties of instances in practice might make them more amenable than their worst-case NP-complete behaviour suggests.^[11] The goal in finding these "hard" instances is for their use in public key cryptography systems, such as the Merkle-Hellman knapsack cryptosystem.

Furthermore, notable is the fact that the hardness of the knapsack problem depends on the form of the input. If the weights and profits are given as integers, it is weakly NP-complete, while it is strongly NP-complete if the weights and profits are given as rational numbers.^[12] However, in the case of rational weights and profits it still admits a fully polynomial-time approximation scheme.

Solving

Several algorithms are available to solve knapsack problems, based on the dynamic programming approach,^[13] the branch and bound approach^[14] or hybridizations of both approaches.^{[11][15][16][17]}

Dynamic programming in-advance algorithm

The **unbounded knapsack problem** (UKP) places no restriction on the number of copies of each kind of item. Besides, here we assume that $x_i > 0$

$$m[w'] = \max \left(\sum_{i=1}^n v_i x_i \right)$$

subject to $\sum_{i=1}^n w_i x_i \leq w'$ and $x_i > 0$

Observe that $m[w]$ has the following properties:

1. $m[0] = 0$ (the sum of zero items, i.e., the summation of the empty set).
2. $m[w] = \max(v_1 + m[w - w_1], v_2 + m[w - w_2], \dots, v_n + m[w - w_n])$, $w_i \leq w$, where v_i is the value of the i -th kind of item.

The second property needs to be explained in detail. During the process of the running of this method, how do we get the weight w ? There are only i ways and the previous weights are $w - w_1, w - w_2, \dots, w - w_i$ where there are total i kinds of different item (by saying different, we mean that the weight and the value are not completely the same). If we know each value of these i items and the related maximum value previously, we just compare them to each other and get the maximum value ultimately and we are done.

Here the maximum of the empty set is taken to be zero. Tabulating the results from $m[0]$ up through $m[W]$ gives the solution. Since the calculation of each $m[w]$ involves examining at most n items, and there are at most W values of $m[w]$ to calculate, the running time of the dynamic programming solution is $O(nW)$. Dividing $w_1, w_2, \dots, w_n, W$ by their greatest common divisor is a way to improve the running time.

Even if $P \neq NP$, the $O(nW)$ complexity does not contradict the fact that the knapsack problem is NP-complete, since W , unlike n , is not polynomial in the length of the input to the problem. The length of the W input to the problem is proportional to the number of bits in W , $\log W$, not to W itself. However, since this runtime is pseudopolynomial, this makes the (decision version of the) knapsack problem a weakly NP-complete problem.

0-1 knapsack problem

A similar dynamic programming solution for the 0-1 knapsack problem also runs in pseudo-polynomial time. Assume $w_1, w_2, \dots, w_n, W$ are strictly positive integers. Define $m[i, w]$ to be the maximum value that can be attained with weight less than or equal to w using items up to i (first i items).

			w							
i	v	w	i	0	1	2	3	4	5	6
1	5	4								
2	4	3								
3	3	2								
4	2	1								
Capacity=6										

A demonstration of the dynamic programming approach.

We can define $m[i, w]$ recursively as follows: **(Definition A)**

- $m[0, w] = 0$
- $m[i, w] = m[i - 1, w]$ if $w_i > w$ (the new item is more than the current weight limit)
- $m[i, w] = \max(m[i - 1, w], m[i - 1, w - w_i] + v_i)$ if $w_i \leq w$.

The solution can then be found by calculating $m[n, W]$. To do this efficiently, we can use a table to store previous computations.

The following is pseudocode for the dynamic program:

```

1 // Input:
2 // Values (stored in array v)
3 // Weights (stored in array w)
4 // Number of distinct items (n)
5 // Knapsack capacity (W)
6 // NOTE: The array "v" and array "w" are assumed to store all relevant values starting at index 1.
7
8 array m[0..n, 0..W];
9 for j from 0 to W do:
10     m[0, j] := 0
11 for i from 1 to n do:
12     m[i, 0] := 0
13
14 for i from 1 to n do:
15     for j from 0 to W do:
16         if w[i] > j then:
17             m[i, j] := m[i-1, j]
18         else:
19             m[i, j] := max(m[i-1, j], m[i-1, j-w[i]] + v[i])

```

This solution will therefore run in $O(nW)$ time and $O(nW)$ space. (If we only need the value $m[n, W]$, we can modify the code so that the amount of memory required is $O(W)$ which stores the recent two lines of the array "m".)

However, if we take it a step or two further, we should know that the method will run in the time between $O(nW)$ and $O(2^n)$. From **Definition A**, we know that there is no need to compute all the weights when the number of items and the items themselves that we chose are fixed. That is to say, the program above computes more than necessary because the weight changes from 0 to W often. From this perspective, we can program this method so that it runs recursively.

```

1 // Input:
2 // Values (stored in array v)
3 // Weights (stored in array w)
4 // Number of distinct items (n)
5 // Knapsack capacity (W)
6 // NOTE: The array "v" and array "w" are assumed to store all relevant values starting at index 1.
7
8 Define value[n, W]
9
10 Initialize all value[i, j] = -1
11
12 Define m:=(i,j) // Define function m so that it represents the maximum value we can get under
the condition: use first i items, total weight limit is j
13 {
14     if i == 0 or j <= 0 then:
15         value[i, j] = 0
16         return
17
18     if (value[i-1, j] == -1) then: // m[i-1, j] has not been calculated, we have to call function m
19         m(i-1, j)
20
21     if w[i] > j then: // item cannot fit in the bag
22         value[i, j] = value[i-1, j]
23     else:
24         if (value[i-1, j-w[i]] == -1) then: // m[i-1, j-w[i]] has not been calculated, we have to
call function m
25             m(i-1, j-w[i])
26         value[i, j] = max(value[i-1, j], value[i-1, j-w[i]] + v[i])
27 }
28
29 Run m(n, W)

```

For example, there are 10 different items and the weight limit is 67. So,

$$w[1] = 23, w[2] = 26, w[3] = 20, w[4] = 18, w[5] = 32, w[6] = 27, w[7] = 29, w[8] = 26, w[9] = 30$$

$$v[1] = 505, v[2] = 352, v[3] = 458, v[4] = 220, v[5] = 354, v[6] = 414, v[7] = 498, v[8] = 545, v[9] :$$

If you use above method to compute for $m(10, 67)$, you will get this, excluding calls that produce $m(i, j) = 0$:

$m(10, 67) = 1270$
 $m(9, 67) = 1270, m(9, 40) = 678$
 $m(8, 67) = 1270, m(8, 40) = 678, m(8, 37) = 545$
 $m(7, 67) = 1183, m(7, 41) = 725, m(7, 40) = 678, m(7, 37) = 505$
 $m(6, 67) = 1183, m(6, 41) = 725, m(6, 40) = 678, m(6, 38) = 678, m(6, 37) = 505$
 $m(5, 67) = 1183, m(5, 41) = 725, m(5, 40) = 678, m(5, 38) = 678, m(5, 37) = 505$
 $m(4, 67) = 1183, m(4, 41) = 725, m(4, 40) = 678, m(4, 38) = 678, m(4, 37) = 505, m(4, 35) = 505$
 $m(3, 67) = 963, m(3, 49) = 963, m(3, 41) = 505, m(3, 40) = 505, m(3, 38) = 505, m(3, 37) = 505$
 $m(2, 67) = 857, m(2, 49) = 857, m(2, 47) = 505, m(2, 41) = 505, m(2, 40) = 505, m(2, 38) = 505$
 $m(1, 67) = 505, m(1, 49) = 505, m(1, 47) = 505, m(1, 41) = 505, m(1, 40) = 505, m(1, 38) = 505$

Besides, we can break the recursion and convert it into a tree. Then we can cut some leaves and use parallel computing to expedite the running of this method.

To find the actual subset of items, rather than just their total value, we can run this after running the function above:

```

1  /**
2   * Returns the indices of the items of the optimal knapsack.
3   * i: We can include items 1 through i in the knapsack
4   * j: maximum weight of the knapsack
5   */
6  function knapsack(i: int, j: int): Set<int> {
7      if i == 0 then:
8          return {}
9      if m[i, j] > m[i-1, j] then:
10         return {i} ∪ knapsack(i-1, j-w[i])
11     else:
12         return knapsack(i-1, j)
13 }
14
15 knapsack(n, W)

```

Meet-in-the-middle

Another algorithm for 0-1 knapsack, discovered in 1974^[18] and sometimes called "meet-in-the-middle" due to parallels to a similarly named algorithm in cryptography, is exponential in the number of different items but may be preferable to the DP algorithm when W is large compared to n . In particular, if the w_i are nonnegative but not integers, we could still use the dynamic programming algorithm by scaling and rounding (i.e. using fixed-point arithmetic), but if the problem requires d fractional digits of precision to arrive at the correct answer, W will need to be scaled by 10^d , and the DP algorithm will require $O(W10^d)$ space and $O(nW10^d)$ time.

algorithm Meet-in-the-middle **is**

input: A set of items with weights and values.
output: The greatest combined value of a subset.

partition the set $\{1..n\}$ into two sets A and B of approximately equal size
compute the weights and values of all subsets of each set

for each subset of A **do**
find the subset of B of greatest value such that the combined weight is less than W

keep track of the greatest combined value seen so far

The algorithm takes $O(2^{n/2})$ space, and efficient implementations of step 3 (for instance, sorting the subsets of B by weight, discarding subsets of B which weigh more than other subsets of B of greater or equal value, and using binary search to find the best match) result in a runtime of $O(n2^{n/2})$. As with the meet in the middle attack in cryptography, this improves on the $O(n2^n)$ runtime of a naive brute force approach (examining all subsets of $\{1\dots n\}$), at the cost of using exponential rather than constant space (see also baby-step giant-step). The current state of the art improvement to the meet-in-the-middle algorithm, using insights from Schroeppel and Shamir's Algorithm for Subset Sum, provides as a corollary a randomized algorithm for Knapsack which preserves the $O^*(2^{n/2})$ (up to polynomial factors) running time and reduces the space requirements to $O^*(2^{0.249999n})$ (see ^[19]Corollary 1.4).

Approximation algorithms

As for most NP-complete problems, it may be enough to find workable solutions even if they are not optimal. Preferably, however, the approximation comes with a guarantee of the difference between the value of the solution found and the value of the optimal solution.

As with many useful but computationally complex algorithms, there has been substantial research on creating and analyzing algorithms that approximate a solution. The knapsack problem, though NP-Hard, is one of a collection of algorithms that can still be approximated to any specified degree. This means that the problem has a polynomial time approximation scheme. To be exact, the knapsack problem has a fully polynomial time approximation scheme (FPTAS).^[20]

Greedy approximation algorithm

George Dantzig proposed a greedy approximation algorithm to solve the unbounded knapsack problem.^[21] His version sorts the items in decreasing order of value per unit of weight, $v_1/w_1 \geq \dots \geq v_n/w_n$. It then proceeds to insert them into the sack, starting with as many copies as possible of the first kind of item until there is no longer space in the sack for more. Provided that there is an unlimited supply of each kind of item, if m is the maximum value of items that fit into the sack, then the greedy algorithm is guaranteed to achieve at least a value of $m/2$.

For the bounded problem, where the supply of each kind of item is limited, the above algorithm may be far from optimal. Nevertheless, a simple modification allows us to solve this case: Assume for simplicity that all items individually fit in the sack ($w_i \leq W$ for all i). Construct a solution S_1 by packing items greedily as long as possible, i.e. $S_1 = \{1, \dots, k\}$ where $k = \max_{1 \leq k' \leq n} \sum_{i=1}^{k'} w_i \leq W$. Furthermore, construct a second solution $S_2 = \{k+1\}$ containing the first item that did not fit. Since $S_1 \cup S_2$ provides an upper bound for the LP relaxation of the problem, one of the sets must have value at least $m/2$; we thus return whichever of S_1 and S_2 has better value to obtain a $1/2$ -approximation.

It can be shown that the average performance converges to the optimal solution in distribution at the error rate $n^{-1/2}$ ^[22]

Fully polynomial time approximation scheme

The fully polynomial time approximation scheme (FPTAS) for the knapsack problem takes advantage of the fact that the reason the problem has no known polynomial time solutions is because the profits associated with the items are not restricted. If one rounds off some of the least significant digits of the profit values then they will be bounded by a polynomial and $1/\epsilon$ where ϵ is a bound on the correctness of the solution. This restriction then means that an algorithm can find a solution in polynomial time that is correct within a factor of $(1-\epsilon)$ of the optimal solution.^[20]

```

algorithm FPTAS is
  input:  $\epsilon \in (0,1]$ 
           a list  $A$  of  $n$  items, specified by their values,  $v_i$ , and weights
  output:  $S'$  the FPTAS solution

   $P := \max \{v_i \mid 1 \leq i \leq n\}$  // the highest item value
   $K := \epsilon \frac{P}{n}$ 

  for  $i$  from 1 to  $n$  do
     $v'_i := \lfloor \frac{v_i}{K} \rfloor$ 
  end for

  return the solution,  $S'$ , using the  $v'_i$  values in the dynamic program outlined above

```

Theorem: The set S' computed by the algorithm above satisfies $\text{profit}(S') \geq (1 - \epsilon) \cdot \text{profit}(S^*)$, where S^* is an optimal solution.

Dominance relations

Solving the unbounded knapsack problem can be made easier by throwing away items which will never be needed. For a given item i , suppose we could find a set of items J such that their total weight is less than the weight of i , and their total value is greater than the value of i . Then i cannot appear in the optimal solution, because we could always improve any potential solution containing i by replacing i with the set J . Therefore, we can disregard the i -th item altogether. In such cases, J is said to **dominate** i . (Note that this does not apply to bounded knapsack problems, since we may have already used up the items in J .)

Finding dominance relations allows us to significantly reduce the size of the search space. There are several different types of dominance relations,^[11] which all satisfy an inequality of the form:

$$\sum_{j \in J} w_j x_j \leq \alpha w_i, \text{ and } \sum_{j \in J} v_j x_j \geq \alpha v_i \text{ for some } x \in \mathbb{Z}_+^n$$

where $\alpha \in \mathbb{Z}_+$, $J \subsetneq N$ and $i \notin J$. The vector x denotes the number of copies of each member of J .

Collective dominance

The i -th item is **collectively dominated** by J , written as $i \ll J$, if the total weight of some combination of items in J is less than w_i and their total value is greater than v_i . Formally,

$$\sum_{j \in J} w_j x_j \leq w_i \text{ and } \sum_{j \in J} v_j x_j \geq v_i \text{ for some } x \in \mathbb{Z}_+^n, \text{ i.e. } \alpha = 1. \text{ Verifying this dominance is}$$

computationally hard, so it can only be used with a dynamic programming approach. In fact, this is equivalent to solving a smaller knapsack decision problem where $V = v_i$, $W = w_i$, and the items are restricted to J .

Threshold dominance

The i -th item is **threshold dominated** by J , written as $i \prec\prec J$, if some number of copies of i are dominated by J . Formally, $\sum_{j \in J} w_j x_j \leq \alpha w_i$, and $\sum_{j \in J} v_j x_j \geq \alpha v_i$ for some $x \in \mathbb{Z}_+^n$ and

$\alpha \geq 1$. This is a generalization of collective dominance, first introduced in^[13] and used in the EDUK algorithm. The smallest such α defines the **threshold** of the item i , written

$t_i = (\alpha - 1)w_i$. In this case, the optimal solution could contain at most $\alpha - 1$ copies of i .

Multiple dominance

The i -th item is **multiply dominated** by a single item j , written as $i \ll_m j$, if i is dominated by some number of copies of j . Formally, $w_j x_j \leq w_i$, and $v_j x_j \geq v_i$ for some $x_j \in \mathbb{Z}_+$ i.e.

$J = \{j\}, \alpha = 1, x_j = \lfloor \frac{w_i}{w_j} \rfloor$. This dominance could be efficiently used during preprocessing

because it can be detected relatively easily.

Modular dominance

Let b be the *best item*, i.e. $\frac{v_b}{w_b} \geq \frac{v_i}{w_i}$ for all i . This is the item with the greatest density of value.

The i -th item is **modularly dominated** by a single item j , written as $i \ll_{\equiv} j$, if i is dominated by j plus several copies of b . Formally, $w_j + tw_b \leq w_i$, and $v_j + tv_b \geq v_i$ i.e.

$J = \{b, j\}, \alpha = 1, x_b = t, x_j = 1$.

Variations

There are many variations of the knapsack problem that have arisen from the vast number of applications of the basic problem. The main variations occur by changing the number of some problem parameter such as the number of items, number of objectives, or even the number of knapsacks.

Multi-objective knapsack problem

This variation changes the goal of the individual filling the knapsack. Instead of one objective, such as maximizing the monetary profit, the objective could have several dimensions. For example, there could be environmental or social concerns as well as economic goals. Problems frequently addressed include portfolio and transportation logistics optimizations.^{[23][24]}

As an example, suppose you ran a cruise ship. You have to decide how many famous comedians to hire. This boat can handle no more than one ton of passengers and the entertainers must weigh less than 1000 lbs. Each comedian has a weight, brings in business based on their popularity and asks for a specific salary. In this example, you have multiple objectives. You want, of course, to maximize the popularity of your entertainers while minimizing their salaries. Also, you want to have as many entertainers as possible.

Multi-dimensional knapsack problem

In this variation, the weight of knapsack item i is given by a D-dimensional vector $\overline{w_i} = (w_{i1}, \dots, w_{iD})$ and the knapsack has a D-dimensional capacity vector $(W_1, \dots, W_D)$. The target is to maximize the sum of the values of the items in the knapsack so that the sum of weights in each dimension d does not exceed W_d .

Multi-dimensional knapsack is computationally harder than knapsack; even for $D = 2$, the problem does not have EPTAS unless $P=NP$.^[25] However, the algorithm in^[26] is shown to solve sparse instances efficiently. An instance of multi-dimensional knapsack is sparse if there is a set $J = \{1, 2, \dots, m\}$ for $m < D$ such that for

every knapsack item i , $\exists z > m$ such that $\forall j \in J \cup \{z\}$, $w_{ij} \geq 0$ and $\forall y \notin J \cup \{z\}$, $w_{iy} = 0$. Such instances occur, for example, when scheduling packets in a wireless network with relay nodes.^[26] The algorithm from^[26] also solves sparse instances of the multiple choice variant, multiple-choice multi-dimensional knapsack.

The IHS (Increasing Height Shelf) algorithm is optimal for 2D knapsack (packing squares into a two-dimensional unit size square): when there are at most five square in an optimal packing.^[27]

Multiple knapsack problem

This variation is similar to the [Bin Packing Problem](#). It differs from the Bin Packing Problem in that a subset of items can be selected, whereas, in the Bin Packing Problem, all items have to be packed to certain bins. The concept is that there are multiple knapsacks. This may seem like a trivial change, but it is not equivalent to adding to the capacity of the initial knapsack. This variation is used in many loading and scheduling problems in Operations Research and has a [Polynomial-time approximation scheme](#).^[28]

Quadratic knapsack problem

The quadratic knapsack problem maximizes a quadratic objective function subject to binary and linear capacity constraints.^[29] The problem was introduced by Gallo, Hammer, and Simeone in 1980,^[30] however the first treatment of the problem dates back to Witzgall in 1975.^[31]

Subset-sum problem

The [subset sum problem](#) is a special case of the decision and 0-1 problems where each kind of item, the weight equals the value: $w_i = v_i$. In the field of [cryptography](#), the term *knapsack problem* is often used to refer specifically to the subset sum problem and is commonly known as one of [Karp's 21 NP-complete problems](#).^[32]

The generalization of subset sum problem is called multiple subset-sum problem, in which multiple bins exist with the same capacity. It has been shown that the generalization does not have an [FPTAS](#).^[33]

Geometric knapsack problem

In the geometric knapsack problem, there is a set of rectangles with different values, and a rectangular knapsack. The goal is to pack the largest possible value into the knapsack.^[34]

See also



- [Bin packing problem](#)
- [Change-making problem](#)
- [Combinatorial auction](#)
- [Combinatorial optimization](#)
- [Continuous knapsack problem](#)
- [Cutting stock problem](#)

- [List of knapsack problems](#)
- [Packing problem](#)

Notes

1. Mathews, G. B. (25 June 1897). "On the partition of numbers" (<http://plms.oxfordjournals.org/content/s1-28/1/486.full.pdf>) (PDF). *Proceedings of the London Mathematical Society*. **28**: 486–490. doi:10.1112/plms/s1-28.1.486 (<https://doi.org/10.1112%2Fplms%2Fs1-28.1.486>).
2. Dantzig, Tobias (2007). *Number : the language of science* (The Masterpiece Science ed.). New York: Plume Book. ISBN 9780452288119.
3. Kellerer, Hans; Pferschy, Ulrich; Pisinger, David (2004). *Knapsack problems* (<https://books.google.com/books?id=u5DB7gck08YC>). Berlin: Springer. p. 449. ISBN 978-3-540-40286-2. Retrieved 5 May 2022.
4. Kellerer, Hans; Pferschy, Ulrich; Pisinger, David (2004). *Knapsack problems* (<https://books.google.com/books?id=u5DB7gck08YC>). Berlin: Springer. p. 461. ISBN 978-3-540-40286-2. Retrieved 5 May 2022.
5. Kellerer, Hans; Pferschy, Ulrich; Pisinger, David (2004). *Knapsack problems* (<https://books.google.com/books?id=u5DB7gck08YC>). Berlin: Springer. p. 465. ISBN 978-3-540-40286-2. Retrieved 5 May 2022.
6. Kellerer, Hans; Pferschy, Ulrich; Pisinger, David (2004). *Knapsack problems* (<https://books.google.com/books?id=u5DB7gck08YC>). Berlin: Springer. p. 472. ISBN 978-3-540-40286-2. Retrieved 5 May 2022.
7. Feuerman, Martin; Weiss, Harvey (April 1973). "A Mathematical Programming Model for Test Construction and Scoring". *Management Science*. **19** (8): 961–966. doi:10.1287/mnsc.19.8.961 (<https://doi.org/10.1287%2Fmnsc.19.8.961>). JSTOR 2629127 (<https://www.jstor.org/stable/2629127>).
8. Skiena, S. S. (September 1999). "Who is Interested in Algorithms and Why? Lessons from the Stony Brook Algorithm Repository". *ACM SIGACT News*. **30** (3): 65–74. CiteSeerX 10.1.1.41.8357 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.41.8357>). doi:10.1145/333623.333627 (<https://doi.org/10.1145%2F333623.333627>). ISSN 0163-5700 (<http://www.worldcat.org/issn/0163-5700>). S2CID 15619060 (<https://api.semanticscholar.org/CorpusID:15619060>).
9. Pisinger, D. 2003. Where are the hard knapsack problems? (<http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.87.7431&rep=rep1&type=pdf>) Technical Report 2003/08, Department of Computer Science, University of Copenhagen, Copenhagen, Denmark.
10. Caccetta, L.; Kulanoot, A. (2001). "Computational Aspects of Hard Knapsack Problems". *Nonlinear Analysis*. **47** (8): 5547–5558. doi:10.1016/s0362-546x(01)00658-7 (<https://doi.org/10.1016%2Fs0362-546x%2801%2900658-7>).
11. Poirriez, Vincent; Yanev, Nicola; Andonov, Rumen (2009). "A hybrid algorithm for the unbounded knapsack problem" (<https://doi.org/10.1016%2Fj.disopt.2008.09.004>). *Discrete Optimization*. **6** (1): 110–124. doi:10.1016/j.disopt.2008.09.004 (<https://doi.org/10.1016%2Fj.disopt.2008.09.004>). ISSN 1572-5286 (<http://www.worldcat.org/issn/1572-5286>). S2CID 8820628 (<https://api.semanticscholar.org/CorpusID:8820628>).
12. Wojtczak, Dominik (2018). "On Strong NP-Completeness of Rational Problems". *International Computer Science Symposium in Russia*. Lecture Notes in Computer Science. **10846**: 308–320. arXiv:1802.09465 (<https://arxiv.org/abs/1802.09465>). doi:10.1007/978-3-319-90530-3_26 (https://doi.org/10.1007%2F978-3-319-90530-3_26). ISBN 978-3-319-90529-7. S2CID 3637366 (<https://api.semanticscholar.org/CorpusID:3637366>).

13. Andonov, Rumen; Poirriez, Vincent; Rajopadhye, Sanjay (2000). "Unbounded Knapsack Problem : dynamic programming revisited". *European Journal of Operational Research*. **123** (2): 168–181. CiteSeerX [10.1.1.41.2135](https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.41.2135) (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.41.2135>). doi:[10.1016/S0377-2217\(99\)00265-9](https://doi.org/10.1016/S0377-2217(99)00265-9) (<https://doi.org/10.1016%2FS0377-2217%2899%2900265-9>).
14. S. Martello, P. Toth, *Knapsack Problems: Algorithms and Computer Implementations*, John Wiley and Sons, 1990
15. S. Martello, D. Pisinger, P. Toth, *Dynamic programming and strong bounds for the 0-1 knapsack problem* (https://www.jstor.org/stable/pdf/2634886.pdf?casa_token=wd6Ykeu46i0AAAAA:egeiDBT-4ijlpmm5GUKA8Ywc67SyXsHHCPFR3QlIL5GGDvx546hVljK9vbeJsgfh_1NyDcH8EMh9TkBoqRHvMHA8xtT4uUzG5x2mZJ4FzCE_wC0mEhp0Tg), *Manag. Sci.*, 45:414–424, 1999.
16. Plateau, G.; Elkihel, M. (1985). "A hybrid algorithm for the 0-1 knapsack problem". *Methods of Oper. Res.* **49**: 277–293.
17. Martello, S.; Toth, P. (1984). "A mixture of dynamic programming and branch-and-bound for the subset-sum problem". *Manag. Sci.* **30** (6): 765–771. doi:[10.1287/mnsc.30.6.765](https://doi.org/10.1287/mnsc.30.6.765) (<https://doi.org/10.1287%2Fmnsc.30.6.765>).
18. Horowitz, Ellis; Sahni, Sartaj (1974), "Computing partitions with applications to the knapsack problem", *Journal of the Association for Computing Machinery*, **21** (2): 277–292, doi:[10.1145/321812.321823](https://doi.org/10.1145/321812.321823) (<https://doi.org/10.1145%2F321812.321823>), hdl:[1813/5989](https://hdl.handle.net/1813%2F5989) (<https://hdl.handle.net/1813%2F5989>), MR [0354006](https://mathscinet.ams.org/mathscinet-getitem?mr=0354006) (<https://mathscinet.ams.org/mathscinet-getitem?mr=0354006>), S2CID [16866858](https://api.semanticscholar.org/CorpusID:16866858) (<https://api.semanticscholar.org/CorpusID:16866858>)
19. Nederlof, Jesper; Węgrzycki, Karol (12 April 2021). "Improving Schroeppel and Shamir's Algorithm for Subset Sum via Orthogonal Vectors". *arXiv:2010.08576* (<https://arxiv.org/abs/2010.08576>) [[cs.DS](https://arxiv.org/archive/cs) (<https://arxiv.org/archive/cs>)].
20. Vazirani, Vijay. *Approximation Algorithms*. Springer-Verlag Berlin Heidelberg, 2003.
21. Dantzig, George B. (1957). "Discrete-Variable Extremum Problems". *Operations Research*. **5** (2): 266–288. doi:[10.1287/opre.5.2.266](https://doi.org/10.1287/opre.5.2.266) (<https://doi.org/10.1287%2Fopre.5.2.266>).
22. Calvin, James M.; Leung, Joseph Y. -T. (1 May 2003). "Average-case analysis of a greedy algorithm for the 0/1 knapsack problem". *Operations Research Letters*. **31** (3): 202–210. doi:[10.1016/S0167-6377\(02\)00222-5](https://doi.org/10.1016/S0167-6377(02)00222-5) (<https://doi.org/10.1016%2FS0167-6377%2802%2900222-5>).
23. Chang, T. J., et al. *Heuristics for Cardinality Constrained Portfolio Optimization* (<http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.109.6698&rep=rep1&type=pdf>). Technical Report, London SW7 2AZ, England: The Management School, Imperial College, May 1998
24. Chang, C. S., et al. "Genetic Algorithm Based Bicriterion Optimization for Traction Substations in DC Railway System (<https://scholarbank.nus.edu.sg/handle/10635/72660>).". In Fogel [102], 11-16.
25. Kulik, A.; Shachnai, H. (2010). "There is no EPTAS for two dimensional knapsack" (https://www.cs.technion.ac.il/~hadas/PUB/multi_knap.pdf) (PDF). *Inf. Process. Lett.* **110** (16): 707–712. CiteSeerX [10.1.1.161.5838](https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.161.5838) (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.161.5838>). doi:[10.1016/j.ipl.2010.05.031](https://doi.org/10.1016/j.ipl.2010.05.031) (<https://doi.org/10.1016%2Fj.ipl.2010.05.031>).
26. Cohen, R. and Grebla, G. 2014. "Multi-Dimensional OFDMA Scheduling in a Wireless Network with Relay Nodes" (http://wimnet.ee.columbia.edu/wp-content/uploads/2013/03/paper_short.pdf). in *Proc. IEEE INFOCOM'14*, 2427–2435.
27. Yan Lan, György Dósa, Xin Han, Chenyang Zhou, Attila Benkő [1] (<https://scholar.google.com/citations?user=txyl5aAAAAAJ>): *2D knapsack: Packing squares*, Theoretical Computer Science Vol. 508, pp. 35–40.

28. Chandra Chekuri and Sanjeev Khanna (2005). "A PTAS for the multiple knapsack problem". *SIAM Journal on Computing*. **35** (3): 713–728. [CiteSeerX 10.1.1.226.3387](https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.226.3387) (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.226.3387>). doi:10.1137/s0097539700382820 (<https://doi.org/10.1137/s0097539700382820>).
29. Wu, Z. Y.; Yang, Y. J.; Bai, F. S.; Mammadov, M. (2011). "Global Optimality Conditions and Optimization Methods for Quadratic Knapsack Problems". *J Optim Theory Appl*. **151** (2): 241–259. doi:10.1007/s10957-011-9885-4 (<https://doi.org/10.1007/s10957-011-9885-4>). S2CID 31208118 (<https://api.semanticscholar.org/CorpusID:31208118>).
30. Gallo, G.; Hammer, P. L.; Simeone, B. (1980). *Quadratic knapsack problems*. *Mathematical Programming Studies*. Vol. 12. pp. 132–149. doi:10.1007/BFb0120892 (<https://doi.org/10.1007/BFb0120892>). ISBN 978-3-642-00801-6.
31. Witzgall, C. (1975). "Mathematical methods of site selection for Electronic Message Systems (EMS)". *NASA Sti/Recon Technical Report N*. NBS Internal report. **76**: 18321. Bibcode:1975STIN...7618321W (<https://ui.adsabs.harvard.edu/abs/1975STIN...7618321W>).
32. Richard M. Karp (1972). "Reducibility Among Combinatorial Problems (<https://web.archive.org/web/20190608032217/https://pdfs.semanticscholar.org/a3c3/7657822859549cd6b12b0d1f76f8ee3680a0.pdf>)". In R. E. Miller and J. W. Thatcher (editors). *Complexity of Computer Computations*. New York: Plenum. pp. 85–103
33. Caprara, Alberto; Kellerer, Hans; Pferschy, Ulrich (2000). "The Multiple Subset Sum Problem" (http://www.or.deis.unibo.it/alberto/mssp_siam.ps). *SIAM J. Optim.* **11** (2): 308–319. [CiteSeerX 10.1.1.21.9826](https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.21.9826) (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.21.9826>). doi:10.1137/S1052623498348481 (<https://doi.org/10.1137/S1052623498348481>).
34. Abed, Fidaa; Chalermsook, Parinya; Correa, José; Karrenbauer, Andreas; Pérez-Lantero, Pablo; Soto, José A.; Wiese, Andreas (2015). *On Guillotine Cutting Sequences* (<https://kops.uni-konstanz.de/handle/123456789/33469>). pp. 1–19. doi:10.4230/LIPIcs.APPROX-RANDOM.2015.1 (<https://doi.org/10.4230/LIPIcs.APPROX-RANDOM.2015.1>). ISBN 978-3-939897-89-7.

References

- Garey, Michael R.; David S. Johnson (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman. ISBN 978-0-7167-1045-5. A6: MP9, pg.247.
- Kellerer, Hans; Pferschy, Ulrich; Pisinger, David (2004). *Knapsack Problems*. Springer. doi:10.1007/978-3-540-24777-7 (<https://doi.org/10.1007/978-3-540-24777-7>). ISBN 978-3-540-40286-2. MR 2161720 (<https://mathscinet.ams.org/mathscinet-getitem?mr=2161720>). S2CID 28836720 (<https://api.semanticscholar.org/CorpusID:28836720>).
- Martello, Silvano; Toth, Paolo (1990). *Knapsack problems: Algorithms and computer implementations* (<http://www.or.deis.unibo.it/knapsack.html>). Wiley-Interscience. ISBN 978-0-471-92420-3. MR 1086874 (<https://mathscinet.ams.org/mathscinet-getitem?mr=1086874>).

External links

- Lecture slides on the knapsack problem (<http://www.cse.unl.edu/~goddard/Courses/CSCE310J/Lectures/Lecture8-DynamicProgramming.pdf>)
- PYAsUKP: Yet Another solver for the Unbounded Knapsack Problem (<https://web.archive.org/web/20111006142943/http://download.gna.org/pyasukp/>), with code taking advantage of the dominance relations in an hybrid algorithm, benchmarks and downloadable copies of some papers.
- Home page of David Pisinger (<http://www.diku.dk/~pisinger/>) with downloadable copies of some papers on the publication list (including "Where are the hard knapsack problems?")

- [Knapsack Problem solutions in many languages \(http://rosettacode.org/wiki/Knapsack_Problem\)](http://rosettacode.org/wiki/Knapsack_Problem) at [Rosetta Code](#)
 - [Dynamic Programming algorithm to 0/1 Knapsack problem \(http://www.personal.kent.edu/~rmuhamma/Algorithms/MyAlgorithms/Dynamic/knapsackdyn.htm\)](http://www.personal.kent.edu/~rmuhamma/Algorithms/MyAlgorithms/Dynamic/knapsackdyn.htm)
 - [Knapsack Problem solver \(online\) \(https://web.archive.org/web/20140223114908/http://karaffeltut.com/NEWKaraffeltutCom/Knapsack/knapsack.html\)](https://web.archive.org/web/20140223114908/http://karaffeltut.com/NEWKaraffeltutCom/Knapsack/knapsack.html)
 - [Solving 0-1-KNAPSACK with Genetic Algorithms in Ruby \(http://www.nils-haldenwang.de/computer-science/computational-intelligence/genetic-algorithm-vs-0-1-knapsack\)](http://www.nils-haldenwang.de/computer-science/computational-intelligence/genetic-algorithm-vs-0-1-knapsack) Archived (<https://web.archive.org/web/20110523210824/http://www.nils-haldenwang.de/computer-science/computational-intelligence/genetic-algorithm-vs-0-1-knapsack>) 23 May 2011 at the Wayback Machine
 - [Codes for Quadratic Knapsack Problem \(http://www.adaptivebox.net/CILib/code/qkpcodes_link.html\)](http://www.adaptivebox.net/CILib/code/qkpcodes_link.html)
-
- [Optimizing Three-Dimensional Bin Packing \(https://web.archive.org/web/20190303205438/http://pdfs.semanticscholar.org/bb99/86af2f26f7726fcef1bc684eac8239c9b853.pdf\)](https://web.archive.org/web/20190303205438/http://pdfs.semanticscholar.org/bb99/86af2f26f7726fcef1bc684eac8239c9b853.pdf)
 - [Knapsack Integer Programming Solution in Python \(http://apmonitor.com/me575/index.php/Main/KnapsackOptimization\)](http://apmonitor.com/me575/index.php/Main/KnapsackOptimization) [Gekko](#) (optimization software)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Knapsack_problem&oldid=1148753573"